QA

QA Terraform in Practice - Intermediate

Terraforming Cloud Platforms
Student Manual

HashiCorp Terraform

Contents

1. Terraform refresher 4
   - Introduction to Terraform 4
   - Key Features of Terraform 4
   - Terraform Workflow 5
   - Lab 1 Sample Deployment 5
2. JSON Syntax Primer 5
   - Introduction to JSON in Terraform 5
   - JSON Structure in Terraform 6
   - Basic Syntax Rules 6
   - Representation of Terraform Blocks in JSON 7
   - Identifiers in JSON-based Terraform Configurations 7
   - Handling Comments in JSON 7
   - Variables and their types in Terraform JSON 7
   - Arguments and Their Role in JSON-based Terraform Configurations 8
   - Lab 2 Using JSON formatted Terraform files 8
3. Cloud Authentication Strategies 9
   - Overview 9

Why Secure Authentication Matters in Terraform? 9
Understanding Cloud Authentication Models 9
Cloud-Specific Implementations 10
Secure Terraform Authentication Across Cloud Providers 11
Terraform Authentication Configuration Examples 12
Recommendation 12
Choosing the Right Credentials Storage 12
Choosing the Best Approach for Multi-Developer Access 16
Lab 3 Cloud Authentication 17
4. Modules, Templates and Code Re-use 17
Introduction to Terraform Modules 17
Benefits of Using Modules 17
Structure of a Terraform Module 17
Creating a Simple Terraform Module 18
Versioning and Module Sources 19
Introduction to Templates in Terraform 19
Lab 4 Modules and Templates 21
5. Implementing Checks and Validations in Terraform 21
Introduction 21
Input Validation in Terraform 22
Preconditions and Postconditions in Terraform 22
Post-Deployment Validation 23
Enforcing Policy with HashiCorp Sentinel 24
Preventing Policy Bypass 24
Best Practise 25
Lab 5 Checks and Validations 25
6. Dynamic Terraform Configurations: Expressions, Loops, and Functions 25
Introduction 25
Expressions in Terraform 26
Types of Expressions 26
Conditional Expressions 26
Loops in Terraform 27

Dynamic Blocks in Terraform 27
Common Functions in Terraform 28
Regular Expressions (Regex) in Terraform 29
Lab 6 Expression and Dynamic Blocks 31
7. Administering Cloud BLOB Storage 31
Introduction 31
Overview of Object Storage in AWS, Azure, and Google Cloud 31
Key Features of Cloud Object Storage 32
Provisioning Object Storage with Terraform 32
Managing Object Storage Configurations Across Clouds 34
Lab 7 Managing Object Storage 36
8. Terraform Pipelining 36
Terraform Community Edition 36
Terraform Enterprise 36
HCP Terraform 37
HCP Terraform Workspaces 38
Terraform in a pipeline 40
Pull request plans and deployment plans 40
A shared Terraform command sequence 40
GitHub Actions 41
Azure DevOps Pipelines 42
Comparing the pipeline options 43
Where HCP Terraform fits 43
9. Bonus Module: Administering Cloud RDBMS 44
Introduction 44
Overview of RDBMS Offerings in AWS, Azure, and Google Cloud 44
Key Features of Cloud RDBMS Solutions 44
Provisioning Relational Databases with Terraform 45
Database Configuration and Management 46
Monitoring & Auditing 47
10. Bonus Module - Kubernetes Management with Terraform 48
Introduction 48

Key Components of Kubernetes 48
Overview of Managed Kubernetes Services (AKS, EKS, GKE) 48
Provisioning Kubernetes Clusters with Terraform 49
Managing Kubernetes Resources with Terraform 50
Multi-Cloud Kubernetes Deployment Considerations 52
Terraform and Kubernetes State Management 52
Introducing Helm charts 53
Appendix. Terraform Associate 004 course coverage 55
Reading the matrix 55
Configuration coverage 57
Modules and state coverage 59
Maintenance and HCP coverage 60
Official references 61

1. Terraform refresher

Introduction to Terraform



Terraform is an open-source Infrastructure as Code (IaC) tool developed by HashiCorp, designed to enable users to define, provision, and manage infrastructure using declarative configuration files. By treating infrastructure as code, Terraform allows for version-controlled, automated, and repeatable deployments across various environments. It supports multiple cloud providers, including Azure, AWS, and Google Cloud, as well as on-premises and hybrid infrastructure solutions. This flexibility makes Terraform an essential tool for organizations aiming to maintain consistency and scalability in their infrastructure management.

Key Features of Terraform

One of Terraform's core strengths is its Infrastructure as Code (IaC) approach, allowing infrastructure configurations to be written in code. This capability facilitates version control, peer reviews, and automated deployments, ensuring consistency across development, testing, and production environments. The declarative approach used by Terraform enables users to define the desired end state of infrastructure, while Terraform determines the necessary steps to achieve that state. Unlike imperative

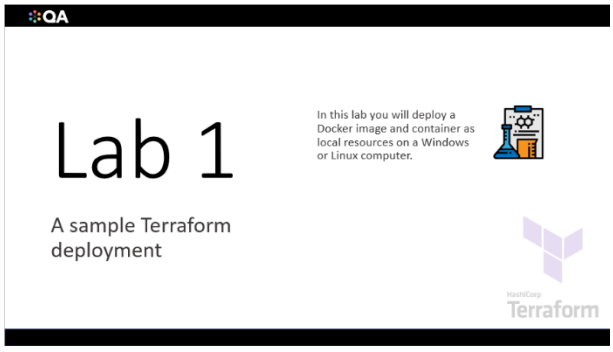
approaches that require step-by-step instructions, Terraform's declarative model simplifies infrastructure management by handling dependencies and order of execution automatically.

Another significant advantage is Terraform's multi-cloud support, which allows users to manage infrastructure across different cloud providers using a single configuration language. This feature eliminates the need for cloud-specific tooling, making it easier to implement hybrid or multi-cloud strategies. Terraform also ensures idempotency, meaning that running the same configuration multiple times results in the same infrastructure state without unintended changes. This behaviour is critical for maintaining consistency in large-scale environments and avoiding configuration drift.

Terraform Workflow

Terraform operates through a structured workflow consisting of four main stages: init, plan, apply, and destroy. The terraform init command initializes the working directory, downloading required providers and modules to set up the environment. The terraform plan command generates an execution plan, showing what changes will be made to align the current infrastructure state with the desired configuration. This step allows users to review and validate modifications before applying them. The terraform apply command executes the planned changes, creating, updating, or deleting resources as defined in the configuration files. Finally, terraform destroy removes infrastructure resources when they are no longer needed, ensuring efficient resource cleanup and cost management.

Lab 1 Sample Deployment



This lab is intended as a refresher to Terraform and guides you through using it to interact with locally installed Docker. It (re)introduces providers, used to indicate the infrastructure that will be managed, and the structure and composition of terraform files. You will review an example terraform file before initializing the client, planning, and then applying the deployment. You will then modify the deployment before eventually destroying it.

2. JSON Syntax Primer

Introduction to JSON in Terraform

JSON (JavaScript Object Notation) is a lightweight, text-based format used for data exchange between systems. It is widely adopted due to its simplicity and ease of parsing, making it a preferred format for automation and machine-readability. In

Terraform, JSON serves as an alternative to HashiCorp Configuration Language (HCL), offering a more rigid structure that aligns well with programmatic automation, templating, and integrations with other tools. While HCL is more human-friendly due to its flexibility and readability, JSON is often used when Terraform configurations need to be dynamically generated or processed by external systems.

JSON Structure in Terraform

A JSON document is composed of key-value pairs, where keys must always be strings enclosed in double quotes, while values can be of different data types such as strings, numbers, booleans, arrays, objects, or null. Objects in JSON are enclosed within curly braces {} and hold multiple key-value pairs, while arrays are enclosed in square brackets [] and store an ordered collection of values. The hierarchical nature of JSON allows complex structures to be defined through nested objects and arrays. This characteristic is particularly useful in Terraform, as it enables detailed infrastructure configurations in a structured manner.

For example, defining an Azure Virtual Network in Terraform using JSON follows this format:

```
{
 "resource": {
  "azurerm_virtual_network": {
   "example": {
    "name": "example-vnet",
    "address_space": ["10.0.0.0/16"],
    "location": "East US",
    "resource_group_name": "example-resources"
   }
  }
 }
}
```

This structure clearly represents the hierarchical relationship between the resource type (azurerm_virtual_network), the unique identifier (example), and its associated configuration properties such as name, address_space, location, and resource_group_name.

Basic Syntax Rules

To ensure a valid JSON configuration, several syntax rules must be followed. First, all keys must be enclosed in double quotes. Second, values can be of different types, including strings, numbers, booleans, arrays, and objects, allowing flexibility in defining

configurations. Third, commas are required to separate key-value pairs within objects and elements within arrays. Missing or misplaced commas often result in JSON parsing errors, making strict adherence to these rules essential.

Representation of Terraform Blocks in JSON

Terraform configurations rely on blocks to define infrastructure components such as resources, modules, and variables. While HCL represents blocks using a structured indentation format, JSON represents them as nested objects, maintaining the hierarchical relationship through proper nesting. For example, a module block in Terraform JSON is represented as an object within the "module" key, containing further nested properties that define its parameters.

Identifiers in JSON-based Terraform Configurations

Identifiers in Terraform JSON files serve as unique names assigned to resources, modules, and variables. These identifiers must be unique within their respective scope to prevent conflicts. For instance, in the previous example, "example" is an identifier for the virtual network resource, distinguishing it from other virtual networks that may exist within the configuration.

Handling Comments in JSON

Unlike HCL, JSON does not natively support comments. This limitation requires developers to find alternative ways to document their configurations, such as using self-explanatory key names, maintaining separate documentation, or embedding metadata within objects. In situations where explanatory context is necessary, structured metadata fields within JSON objects can be used to store descriptive information.

Variables and their types in Terraform JSON

Terraform supports multiple variable types that allow dynamic customization of configurations. These include strings for text-based values, numbers for numeric entries, booleans for true/false logic, lists (arrays) for ordered collections of elements, and maps (objects) for key-value pairs. Variables in Terraform JSON are typically defined under the "variable" key, with each variable name as a sub-key. For example, defining an Azure Resource Group name as a variable in JSON follows this format:

```
{
 "variable": {
  "resource_group_name": {
   "default": "example-resources"
  }
 }
```

```
}
```

This structure allows the variable "resource_group_name" to be referenced dynamically within the configuration.

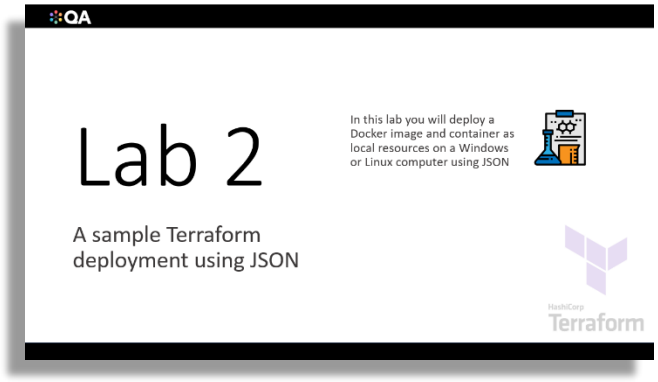
Arguments and Their Role in JSON-based Terraform Configurations

Arguments in Terraform JSON are key-value pairs used to configure the properties of resources. These arguments define essential attributes such as the name, address space, location, and resource group for a given resource. The following example illustrates how arguments are used within a Terraform JSON configuration for an Azure Virtual Network:

```
{
 "resource": {
  "azurerm_virtual_network": {
   "example": {
    "name": "example-vnet",
    "address_space": ["10.0.0.0/16"],
    "location": "East US",
    "resource_group_name": "example-resources"
   }
  }
 }
}
```

Each argument, such as "name" and "address_space", specifies configuration details required for proper deployment.

Lab 2 Using JSON formatted Terraform files



This lab will compare a Terraform HCL (HashiCorp Configuration Language) configuration with a JSON formatted terraform file

3. Cloud Authentication Strategies

Overview

When using Terraform and automation tools in cloud environments, securely managing authentication credentials is critical. Cloud providers offer different authentication mechanisms and understanding when to use Service Principals (SPs) / Service Accounts versus Managed Identity offerings (or their equivalents) is essential.

Cloud authentication strategies must balance security, ease of management, and automation capabilities. Service Principals and Service Accounts are widely used for CI/CD pipelines and external clients, while Managed Identities, Instance Profiles, and Workload Identity Federation eliminate the need for credential management but are limited to internal workloads.

This module explores authentication best practices across Azure, AWS, and Google Cloud, comparing Service Principals vs. Managed Identities, discussing cloud-specific implementations, and providing recommendations for secure Terraform authentication.

Why Secure Authentication Matters in Terraform?

Terraform requires authentication credentials to provision cloud infrastructure. Hardcoding credentials in configuration files or environment variables introduces security risks, making systems vulnerable to credential leaks or unauthorized access. Using cloud-native authentication mechanisms ensures that credentials remain secure, minimizing manual management while enforcing access control policies.

Cloud providers offer native services for securely storing and retrieving authentication credentials, reducing exposure risks. These services allow Terraform to authenticate dynamically, eliminating static credential storage. Organizations must implement role-based access controls, least privilege principles, and audit logging to enforce security and compliance.

Understanding Cloud Authentication Models

Each major cloud provider offers two primary authentication models:

Service Principals / Service Accounts

Service Principals (Azure), IAM Users/Roles (AWS), and Service Accounts (Google Cloud) are used to authenticate applications, CI/CD pipelines, and external automation. These identities are non-human accounts that allow Terraform and other automation tools to authenticate securely.

To use a Service Principal or Service Account, a set of credentials is generated, usually consisting of an ID and secret key. These credentials must be stored securely, ideally in

a secret management solution. The main advantage of this model is its flexibility, as Service Principals and Service Accounts can be used in any environment, including multi-cloud and hybrid setups.

However, one major drawback is credential management. Since these credentials do not automatically rotate, they must be manually rotated or stored in a system that supports automatic key rotation, such as Azure Key Vault, AWS Secrets Manager, or Google Cloud Secret Manager.

Managed Identities / Instance Profiles / Workload Identity Federation

Managed Identities (Azure), Instance Profiles (AWS), and Workload Identity Federation (Google Cloud) offer seamless authentication for workloads running inside the cloud provider's infrastructure. These services eliminate the need for storing or rotating credentials manually, as authentication happens transparently through IAM policies.

Managed Identities work by associating an identity with a cloud resource (such as a Virtual Machine or Function App). This identity is then granted the necessary permissions to authenticate with cloud services. Similarly, AWS Instance Profiles allow EC2 instances and Lambda functions to assume roles with defined permissions, while Google's Workload Identity Federation enables Kubernetes workloads to authenticate without using long-lived credentials.

While these solutions significantly improve security by eliminating static credentials, they are limited in scope. They cannot be used for CI/CD pipelines, external clients, or cross-cloud automation. These solutions are ideal for internal cloud workloads but do not replace the need for Service Principals or Service Accounts in broader authentication use cases.

Cloud-Specific Implementations

Azure: Service Principals vs. Managed Identities

Azure provides two primary authentication mechanisms:

- **Service Principals (SPs)**: Used for CI/CD pipelines, automation, and cross-cloud authentication. Requires explicit credential management.
- **Managed Identities**: Designed for internal workloads within Azure, eliminating the need for credential storage.

A Service Principal in Azure acts as an identity that Terraform, or other tools can use to authenticate with Azure resources. These are typically stored in Azure Key Vault for security. In contrast, Managed Identities are automatically handled by Azure Active Directory and can be assigned to Virtual Machines, App Services, and Kubernetes workloads without requiring explicit credentials.

AWS: IAM Users/Roles vs. Instance Profiles/Federated Identities

AWS offers authentication options tailored to specific use cases:

- **IAM Users & IAM Roles**: Equivalent to Service Principals, best suited for CI/CD pipelines and external automation.
- **Instance Profiles & IAM Roles for AWS Services**: Used for internal workloads, such as EC2 instances and Lambda functions.

For external automation, IAM Users or Roles can be configured with access keys that are stored securely in AWS Secrets Manager. For internal workloads, AWS recommends using Instance Profiles, where credentials are provided dynamically through AWS STS (Security Token Service), ensuring security, and reducing operational overhead.

Google Cloud: Service Accounts vs. Workload Identity Federation

Google Cloud provides:

- **Service Accounts**: Like Service Principals, used for CI/CD and external automation.
- **Workload Identity Federation**: Designed for internal Google Cloud workloads, removing the need for JSON keys.

Google Cloud Service Accounts work well for Terraform and automation but require proper key management practices to prevent exposure. Workload Identity Federation allows Kubernetes workloads to authenticate using external identity providers without requiring a stored key.

Secure Terraform Authentication Across Cloud Providers

Organizations should follow **best practices** to ensure Terraform authentication is secure:

1. **Store credentials securely** in a managed secret management solution (e.g., AWS Secrets Manager, Azure Key Vault, or Google Cloud Secret Manager).
2. **Use short-lived credentials** where possible (AWS STS, Google Workload Identity Federation) to reduce the attack surface.
3. **Enforce RBAC policies** to ensure Terraform only has access to necessary resources.
4. **Monitor and audit authentication events** to detect anomalies.

Terraform Authentication Configuration Examples

Each cloud provider supports retrieving authentication credentials dynamically

AWS Example using Secrets Manager

```
provider "aws" {
 access_key =
data.aws_secretsmanager_secret_version.access_key.secret_string
 secret_key =
data.aws_secretsmanager_secret_version.secret_key.secret_string
 region    = "us-east-1"
}
```

Azure Example using Key Vault

```
provider "azurerm" {
 client_id      = data.azurerm_key_vault_secret.client_id.value
 client_secret  = data.azurerm_key_vault_secret.client_secret.value
 tenant_id      = data.azurerm_key_vault_secret.tenant_id.value
 subscription_id = "your-subscription-id"
 features {}
}
```

Google Cloud Example using Secret Manager

```
provider "google" {
 credentials = file(var.gcp_credentials)
 project    = "your-project-id"
 region     = "us-central1"
}
```

Recommendation

Organizations should use Service Principals, IAM Users, or Service Accounts for CI/CD and automation pipelines, ensuring that credentials are securely stored. Managed Identities, Instance Profiles, and Workload Identity Federation should be used for internal workloads to avoid credential management. Combining these authentication methods allows for a secure and scalable approach to Terraform and cloud automation.

Choosing the Right Credentials Storage

When managing Service Principal (SP) credentials for Terraform in a multi-developer environment, it is crucial to strike a balance between security, accessibility, and persistence. The table below outlines different ways to store credentials, each with

distinct security implications. Below this, we explore these methods in more detail and discuss their applicability to secure SP usage across multiple developers.

Storage Methods for Service Principal (SP) Credentials

Storage Method	Persists ?	Scope	Requires Admin?
Temporary Environment Variables	No	Current session only	No
User Level Environment Variables	Yes	Only current user	No
Machine Level Environment Variables	Yes	System-wide	Yes
PowerShell/Bash Script ()	No	Run manually	No
.env File	No	Run manually	No
Azure Key Vault	Yes	Centralized, cloud-managed	No
AWS Secrets Manager	Yes	Centralized, cloud-managed	No
GCP Secret Manager	Yes	Centralized, cloud-managed	No

Session-Only Environment Variables (Temporary Variables)

Method: Exporting credentials dynamically into environment variables via shell commands (e.g., export VAR_NAME="value" in Bash, env:VAR_NAME = "value" in PowerShell) or using cloud SDK authentication commands.

Example:

- **Bash:** export ARM_CLIENT_ID=<client_id>
- **PowerShell:** $env:ARM_CLIENT_ID = "<client_id>"
- **Azure CLI:** az login --service-principal -u <client_id> -p <client_secret> --tenant <tenant_id>
- **AWS CLI:** aws configure set aws_access_key_id <access_key>
- **GCP CLI:** gcloud auth application-default login

Security Considerations: This is a safe approach for short-term use because credentials disappear when the session ends. However, it does not scale well for multiple developers, as each developer must manually re-enter the credentials in each session

Best for: One-time, ad-hoc Terraform authentication where security is a priority over convenience.

User-Level Persistent Variables

Method: Storing credentials persistently for an individual user using environment variables.

Example:

- **Bash:** echo 'export ARM_CLIENT_ID="<client_id>"' >> ~/.bashrc
- **PowerShell:** [System.Environment]::SetEnvironmentVariable("ARM_CLIENT_ID", "<client_id>", "User")

Security Considerations: This method provides better usability but has some security risks. While credentials are stored in the Windows environment, they may be accessible to scripts or malware running under the same user account. If a developer's machine is compromised, credentials could be exposed.

Best for: Individual developers who need persistent credentials without affecting other users.

Machine-Level Persistent Variables

Method: Storing credentials system-wide for all users.

Example:

- **Bash:** echo 'export ARM_CLIENT_ID="<client_id>"' | sudo tee -a /etc/environment
- **PowerShell (Admin):** [System.Environment]::SetEnvironmentVariable("ARM_CLIENT_ID", "<client_id>", "Machine")

Security Considerations: High-risk method unless tightly controlled. If a machine is compromised, all users can access the credentials, making this an attractive target for attackers. This should only be used in hardened, dedicated build servers where access is restricted.

Best for: CI/CD pipelines or Terraform automation servers. Not recommended for individual developer machines.

Cloud-Specific Authentication Methods (Recommended for Teams)

Method: Using cloud-native secret storage solutions to securely store and retrieve credentials dynamically.

Example:

- **Azure Key Vault:** az keyvault secret show --name "SP-SECRET" --vault-name "MyKeyVault"
- **AWS Secrets Manager:** aws secretsmanager get-secret-value --secret-id MySecret
- **GCP Secret Manager:** gcloud secrets versions access latest --secret=MY_SECRET

Security Considerations: Credentials are stored securely in a managed vault, but using this method requires additional permissions and infrastructure setup to ensure proper access control and governance.

Best for: Teams needing a secure and scalable solution for multi-developer environments.

PowerShell/Bash Script Authentication

Method: Using a script to dynamically set environment variables for authentication.

Example:

- **PowerShell:** set-env.ps1

  ```
  $env:ARM_CLIENT_ID = "<client_id>"
  $env:ARM_CLIENT_SECRET = "<client_secret>"
  ```

- **Bash:** set-env.sh

  ```
  export ARM_CLIENT_ID="<client_id>"
  export ARM_CLIENT_SECRET="<client_secret>"
  ```

Security Considerations: Since credentials are not stored permanently, this method is more secure than persistent environment variables. However, developers must ensure that the script itself is not compromised and should avoid committing it to version control (e.g., use .gitignore).

Best for: Teams that need a repeatable but temporary way to authenticate Terraform.

.env File (Manually Loaded)

Method: Storing credentials in a plaintext .env file that can be sourced when needed.

Example:

- .env file:

  ```
  ARM_CLIENT_ID=<client_id>
  ARM_CLIENT_SECRET=<client_secret>
  ```

- **Bash:** `source .env`
- **PowerShell:** `Get-Content .env | ForEach-Object { $name, $value = $_.split("="); Set-Item -Path env:$name -Value $value }`

Security Considerations: If an .env file is leaked, an attacker can obtain full access to the Service Principal. This method should only be used in controlled environments where access to the file is strictly restricted. An alternative is to store .env files in a secure secrets management tool (e.g., Azure Key Vault, AWS Secrets Manager).

Best for: CI/CD pipelines where strict file security is enforced. Never use in shared development environments.

Choosing the Best Approach for Multi-Developer Access

For Individual Developers:

- Use **User-Level Persistent Variables** for convenience.
- Use a **PowerShell/Bash Script** for better security.

For Teams in a Shared Environment:

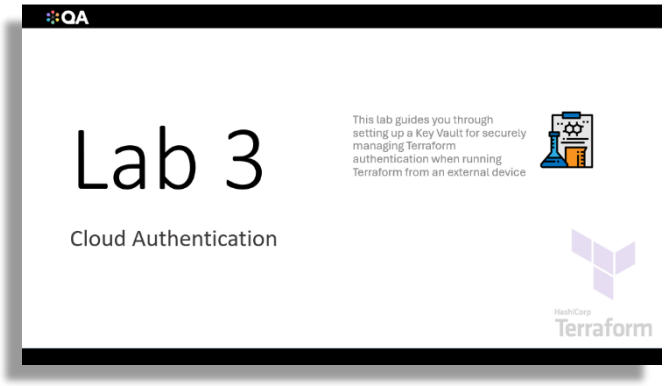
- Use **Cloud-Native Secret Management (Azure Key Vault, AWS Secrets Manager, GCP Secret Manager)** to centrally store and retrieve credentials.
- Avoid **Machine-Level Persistent Variables**, unless in a dedicated CI/CD machine.

For CI/CD Pipelines:

- Use **Machine-Level Persistent Variables** or dynamically inject secrets from a secure store.
- Avoid .env files unless strict access controls are enforced.

By leveraging cloud-native authentication and dynamic credential retrieval, teams can enhance security while maintaining ease of access in Terraform workflows.

Lab 3 Cloud Authentication



This lab guides you through setting up a Cloud specific Key Vault for securely managing Terraform authentication when running Terraform from an external device

4. Modules, Templates and Code Re-use

Introduction to Terraform Modules

Modules in Terraform are self-contained packages of Terraform configurations that help organize and manage infrastructure by grouping related resources together. A module can consist of multiple Terraform configuration files, including resources, input variables, output values, and even nested sub-modules. By using modules, infrastructure teams can standardize deployments, improve reusability, and simplify complex configurations. Instead of defining similar infrastructure components repeatedly across different projects, modules allow teams to write configuration once and reuse it efficiently.

Benefits of Using Modules

Terraform modules offer several key advantages. One of the primary benefits is reusability, as modules can be utilized across multiple projects, reducing the need to rewrite code for similar infrastructure components. This not only saves time but also enhances maintainability. Consistency is another major advantage, as standardizing module configurations minimizes the risk of configuration drift, ensuring that infrastructure remains uniform across different environments. Additionally, modules contribute to simplification by breaking down complex configurations into smaller, manageable components, making it easier to understand, test, and troubleshoot infrastructure code.

Structure of a Terraform Module

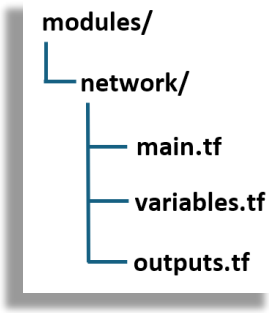
A basic Terraform module typically consists of three primary files:

main.tf – Defines the resources and configurations within the module.

variables.tf – Declares input variables that allow customization of the module.

outputs.tf – Defines output values, which can be referenced by other modules or configurations.

A typical directory structure for a module looks like this:



This structure ensures clear separation of concerns, making it easy to reuse, update, and manage the module.

Creating a Simple Terraform Module

To illustrate how modules work, consider a module that provisions an Azure Virtual Network.

Defining the Module

In the **main.tf** file, the virtual network is created using input variables for flexibility:

```
resource "azurerm_virtual_network" "vnet" {
 name = var.vnet_name
 address_space = var.address_space
 location = var.location
 resource_group_name = var.resource_group_name
}
```

The **variables.tf** file defines the necessary input parameters:

```
variable "vnet_name" {}
variable "address_space" {}
variable "location" {}
variable "resource_group_name" {}
```

The **outputs.tf** file provides an output value, making the virtual network ID accessible to other configurations:

```
output "vnet_id" {
 value = azurerm_virtual_network.vnet.id
```

```
}
```

Using the Module in a Root Configuration

Modules are referenced in a root Terraform configuration using the module block. The example below shows how the previously created module can be used in main.tf:

```
module "network" {
 source          = "./modules/network"
 vnet_name       = "example-vnet"
 address_space   = ["10.0.0.0/16"]
 location        = "East US"
 resource_group_name = "example-resources"
}
```

Input and Output Variables in Modules

Terraform modules support input variables, which allow customization based on different environments or requirements. These variables enable users to pass values dynamically instead of hardcoding them within the module. Output variables are used to expose key information, such as resource IDs, so that other modules or configurations can reference them. Properly structuring input and output variables enhances module flexibility and interoperability.

Versioning and Module Sources

Terraform supports different methods for sourcing modules, enabling flexibility in how they are referenced and maintained:

Local Modules – Referenced using relative paths (e.g., ./modules/network), typically used within a single project.

Remote Modules – Hosted in Terraform Registry, GitHub, or private repositories, allowing for better version control and sharing across teams.

Best Practices – Using version constraints (version = ">= 1.0.0") helps manage updates and ensures module stability without unexpected changes.

Introduction to Templates in Terraform

Terraform also supports templates, which allow for the dynamic generation of content such as configuration files, resource names, and startup scripts. By using templates, users can reduce redundancy and enhance the adaptability of Terraform configurations. Templates work well for generating instance initialization scripts, custom resource names, or structured configuration files.

Working with the templatefile Function

The templatefile function in Terraform reads a template file and replaces placeholders with variable values. This is particularly useful when configuring virtual machines, as it allows user-defined startup scripts to be passed dynamically.

For example, consider the following template file init.tmpl that sets a hostname and displays a welcome message:

```
#!/bin/bash
hostnamectl set-hostname ${hostname}
echo "Welcome to ${environment} environment" > /etc/motd
```

The corresponding Terraform configuration would use the templatefile function to substitute variables:

```
resource "azurerm_virtual_machine" "example" {
 name            = "example-vm"
 location        = var.location
 resource_group_name = var.resource_group_name
 vm_size         = "Standard_DS1_v2"

 os_profile {
  computer_name  = "example"
  admin_username = "azureuser"
  custom_data    = templatefile("init.tmpl", {
   hostname    = "example-vm"
   environment = "production"
  })
 }
}
```

This setup ensures that when the VM is created, it receives a unique hostname and a customized welcome message.

Template Variables and Interpolation

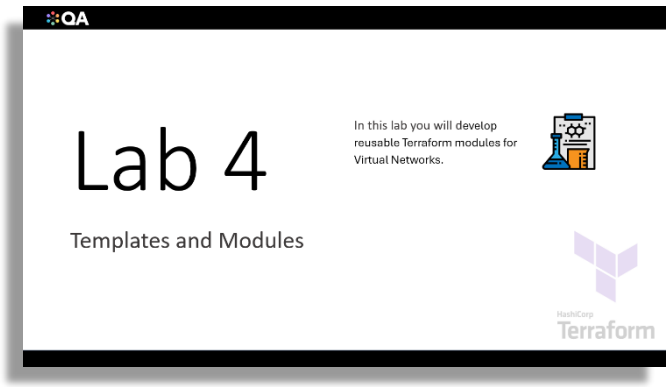
Terraform templates use interpolation to insert variable values dynamically. Placeholders are defined using the ${} syntax, allowing the inclusion of both simple and complex data types, such as strings, lists, and maps. This method enhances modularity and prevents hardcoded values from being embedded directly in Terraform configurations.

Best Practices for Using Modules and Templates

To maximize the effectiveness of Terraform modules and templates, consider the following best practices:

- Use consistent naming conventions for variables to maintain clarity across different modules.
- Keep templates and modules modular and organized, separating resources logically to improve maintainability.
- Document complex templates and modules to facilitate better collaboration within teams.
- Store modules and templates in version-controlled repositories, such as GitHub or a private registry, ensuring proper tracking and collaboration.

Lab 4 Modules and Templates



In this lab you will develop reusable Terraform modules for Virtual Networks. You will parameterize the module using variables and reuse the module with different parameters.

5. Implementing Checks and Validations in Terraform

Introduction

Ensuring infrastructure as code (IaC) deployments adhere to organizational policies and best practices is crucial in cloud environments. Terraform provides mechanisms to enforce validation rules at various stages of the deployment process, preventing misconfigurations and ensuring compliance. This chapter explores input validation, precondition and postcondition checks, post-deployment validation, and external policy enforcement using HashiCorp Sentinel.

Input Validation in Terraform

Input validation ensures that variable values meet specific criteria before Terraform executes a plan or applies changes. It prevents invalid configurations from being deployed, reducing the risk of failure and misconfiguration.

Example: Validating Naming Conventions

Terraform allows validation rules to be applied to input variables using the validation block:

```
variable "resource_group_name" {
 description = "Name of the resource group."
 type        = string
 validation {
  condition     = can(regex("^RG-[1-6]$", var.resource_group_name))
  error_message = "Resource group name must be RG- followed by a number
from 1 to 6."
 }
}
```

This ensures that the resource group name follows the required naming convention (RG-1 to RG-6).

Example: Enforcing Region Selection

```
variable "location" {
 description = "Cloud region for resource deployment."
 type        = string
 validation {
  condition     = contains(["us-east-1", "us-west-2", "europe-west1"],
var.location)
  error_message = "Location must be one of the predefined regions."
 }
}
```

This condition restricts users to specific cloud regions, ensuring compliance with organizational policies.

Preconditions and Postconditions in Terraform

Terraform allows additional validation through precondition and postcondition blocks, enabling checks at runtime to enforce constraints before and after resource creation.

Example: Ensuring a Storage Account Follows Naming Standards

```
resource "azurerm_storage_account" "example" {
```

```
  name                = var.storage_account_name
  resource_group_name = var.resource_group_name
  location            = var.location
  account_tier        = "Standard"
  account_replication_type = "LRS"

  lifecycle {
    precondition {
      condition     = startswith(var.storage_account_name, "stg")
      error_message = "Storage account name must start with 'stg'."
    }
    postcondition {
      condition     = length(self.name) <= 24
      error_message = "Storage account name must be 24 characters or fewer."
    }
  }
}
```

This configuration enforces naming policies and prevents Terraform from creating resources that violate those rules.

Post-Deployment Validation

Validation should not stop at Terraform's internal mechanisms. Some misconfigurations may only become evident after deployment. Cloud platforms provide policy enforcement tools that can be used in combination with Terraform.

Example: Using Policy for Post-Deployment Checks

Cloud policy mechanisms can be used to enforce resource compliance after deployment. For example, Azure Policy can check for compliance and, if a resource violates policy, the deployment is rejected or flagged for review.

```
{
  "properties": {
    "displayName": "Enforce Storage Account Naming Policy",
    "policyType": "Custom",
    "mode": "All",
    "parameters": {},
    "policyRule": {
      "if": {
        "allOf": [
          {
            "field": "name",
```

```
        "notLike": "stg*"
       },
       {
        "field": "type",
        "equals": "Microsoft.Storage/storageAccounts"
       }
      ]
     },
     "then": {
      "effect": "deny"
     }
    }
   }
}
```

This ensures that all storage accounts conform to naming conventions, even if Terraform validation was bypassed.

Enforcing Policy with HashiCorp Sentinel

HashiCorp Sentinel is a policy-as-code framework that provides fine-grained control over Terraform deployments. It allows organizations to define rules that must be met before Terraform applies any changes.

Example: Sentinel Policy to Restrict VM Naming

```
import "tfplan/v2" as tfplan
main = rule {
 all tfplan.resources.azurerm_virtual_machine as _, instances {
  all instances as _, r {
   r.name matches "^VM[1-6]$"
  }
 }
}
```

This policy ensures that virtual machines are named according to the VM1 to VM6 format.

Preventing Policy Bypass

One challenge of using validation within Terraform files is that developers can modify the rules. To prevent policy bypass, organizations should enforce governance through:

- **Terraform Cloud/Enterprise Sentinel Policies**: Ensuring policies run at the organization level.

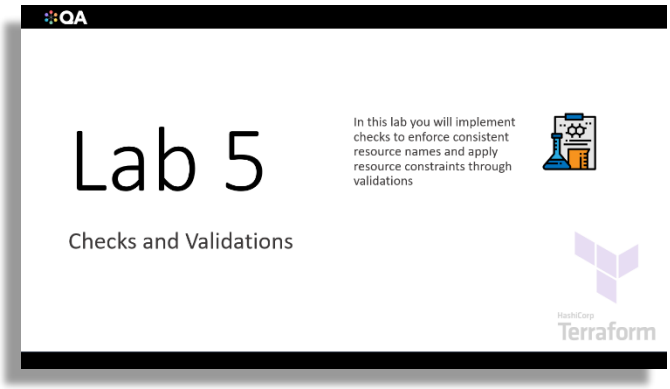
- **Azure Policy, AWS Config Rules, or GCP Organization Policies**: Blocking non-compliant resources at the cloud provider level.
- **Code Reviews and Change Management**: Implementing peer reviews before merging infrastructure changes.

Best Practise

Implementing checks and validations in Terraform improves security, consistency, and governance of cloud deployments. By combining Terraform's built-in validation, cloud-native policy enforcement, and external policy frameworks like Sentinel, organizations can ensure that their infrastructure remains compliant and robust.

- Use Terraform variable validation to enforce input constraints.
- Apply precondition and postcondition checks to validate resources dynamically.
- Implement post-deployment checks using cloud-native policies (Azure Policy, AWS Config, GCP Organization Policies).
- If available, use Sentinel for policy-as-code enforcement at the organization level.
- Combine validation strategies to prevent misconfigurations and unauthorized changes.

Lab 5 Checks and Validations



This lab takes you through deploying resources whilst enforcing naming and policy constraints

6. Dynamic Terraform Configurations: Expressions, Loops, and Functions

Introduction

Expressions, conditional logic, loops, dynamic blocks, and built-in functions provide powerful mechanisms for writing efficient and reusable Terraform configurations. By

leveraging these capabilities, you can create adaptable infrastructure templates that respond dynamically to different environments, input parameters, and deployment scenarios.

Terraform provides a rich set of features for dynamically configuring infrastructure, enabling modular, reusable, and efficient deployments. Expressions, conditional logic, loops, dynamic blocks, and built-in functions allow Terraform configurations to adapt based on input variables, compute derived values, and simplify complex configurations. This chapter explores these capabilities, illustrating their applications in Terraform-based infrastructure management

Expressions in Terraform

Expressions in Terraform enable configurations to be flexible and data-driven. Instead of hardcoding values, users can compute values dynamically using variables, functions, and resource attributes.

Types of Expressions

- **Literal expressions**: Explicitly define static values, e.g., "eastus", 10, true.
- **Interpolation expressions**: Use ${} syntax to reference variables and functions, e.g., ${var.location}.
- **Computed expressions**: Combine values dynamically, e.g., ${var.instance_type == "prod" ? "Standard_D4_v2" : "Standard_B2s"}.
- **Complex expressions**: Use built-in functions for data manipulation, such as join(), split(), and regex().

Conditional Expressions

Conditional expressions allow configurations to be dynamic based on specific conditions. The syntax follows: condition ? true_value : false_value

Example: Setting an Instance Type Based on Environment

```
output "instance_type" {
 value = var.env == "prod" ? "Standard_D4_v2" : "Standard_B2s"
}
```

If var.env is "prod", Terraform assigns "Standard_D4_v2"; otherwise, it defaults to "Standard_B2s". This technique is useful for adjusting configurations based on environment-specific settings without modifying the code manually.

Example: Toggling Resource Creation

```
resource "azurerm_virtual_machine" "example" {
```

```
 count = var.create_vm ? 1 : 0
 name  = "example-vm"
}
```

Terraform only creates the VM if var.create_vm is true.

Loops in Terraform

Terraform provides two looping mechanisms for dynamically generating multiple resources:

Using for_each

The for_each loop iterates over maps and sets.

```
variable "vm_names" {
 type    = set(string)
 default = ["vm1", "vm2", "vm3"]
}

resource "azurerm_virtual_machine" "example" {
 for_each = var.vm_names
 name     = each.key
}
```

Terraform creates a separate VM for each name in vm_names.

Using count

The count parameter controls how many instances of a resource are created, indexed numerically.

```
variable "storage_count" {
 default = 3
}

resource "azurerm_storage_account" "example" {
 count = var.storage_count
 name  = "storage${count.index}"
}
```

Terraform creates three storage accounts (storage0, storage1, storage2).

Dynamic Blocks in Terraform

Dynamic blocks generate nested configurations dynamically based on input data.

Syntax of a Dynamic Block

```
dynamic "block_name" {
 for_each = variable_or_list
 content {
  attribute = block_name.value.key
 }
}
```

Example: Assigning Multiple Security Rules Dynamically

```
variable "security_rules" {
 type = list(object({
  name     = string
  priority  = number
  direction = string
 }))
 default = [
  { name = "allow-ssh", priority = 100, direction = "Inbound" },
  { name = "allow-http", priority = 200, direction = "Inbound" }
 ]
}

resource "azurerm_network_security_group" "example" {
 name = "example-nsg"

 dynamic "security_rule" {
  for_each = var.security_rules
  content {
   name     = security_rule.value.name
   priority  = security_rule.value.priority
   direction = security_rule.value.direction
  }
 }
}
```

This generates security rules dynamically based on input data.

Common Functions in Terraform

Terraform includes various built-in functions to manipulate and transform data. These functions are categorized into:

String Functions

- **join(separator, list):** Concatenates elements with a separator.

```
output "joined_names" {
 value = join("-", ["azure", "resource", "group"])
}
```

Output: "azure-resource-group"

split(separator, string): Splits a string into a list.

```
output "split_names" {
 value = split("-", "azure-resource-group")
}
```

Output: ["azure", "resource", "group"]

Numeric Functions

max() and min(): Return the maximum or minimum value.

```
output "max_value" {
 value = max(10, 20, 15)
}
```

Output: 20

Date/Time Functions

timestamp(): Returns the current UTC time in ISO 8601 format.

```
output "current_timestamp" {
value = timestamp()
}
```

Output: "2025-02-17T12:00:00Z" (example output)

Filesystem Functions

file(filepath): Reads the content of a specified file.

```
output "file_content" {
 value = file("./config.txt")
}
```

This function is useful for injecting configuration data into Terraform resources.

Regular Expressions (Regex) in Terraform

Regular expressions (Regex) are powerful string-matching patterns used for searching, extracting, and manipulating text. They allow Terraform users to define complex search

criteria to validate input, extract values, or transform text dynamically. Regex is particularly useful in scenarios where resource names follow a structured pattern, and users need to enforce consistency or extract meaningful segments from text-based attributes.

Terraform supports regex functions that help match and process strings based on defined patterns. These functions provide flexibility in handling data, ensuring that configurations adhere to expected formats and reducing manual string manipulation efforts.

These examples demonstrate how regex functions can be applied to extract structured information, validate input, and enforce naming conventions dynamically in Terraform.

Extracting a numerical ID from a resource name

```
output "resource_id" {
 value = regex("id-(\d+)", "resource-id-12345")
}
```

Output: "12345"

Validating an email format

```
output "is_valid_email" {
 value = can(regex("^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$",
"user@example.com"))
}
```

Output: true

Extracting multiple region names from a string

```
output "regions" {
 value = regexall("(us-east-1|us-west-2|eu-central-1)", "deploy-us-east-1-us-
west-2")
}
```

Output: ["us-east-1", "us-west-2"]

Extract the first match of a pattern with regex

```
output "environment" {
 value = regex("(prod|dev|test)", "stg-prod-finance-eastus")
}
```

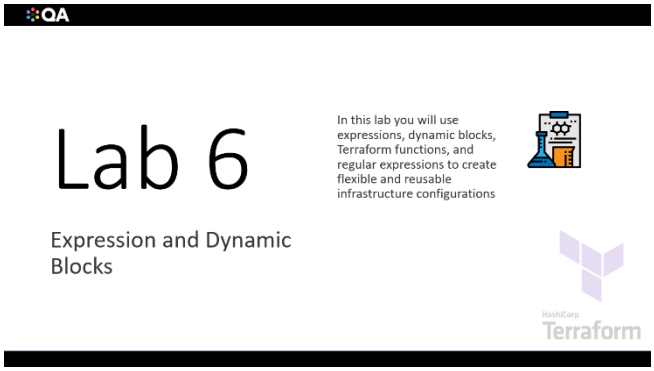
Output: "prod"

Extracting all matches with regexall

```
output "all_matches" {
 value = regexall("(eastus|westus|centralus)", "prod-finance-eastus-westus")
}
```

Output: ["eastus", "westus"]

Lab 6 Expression and Dynamic Blocks



In this lab you will use expressions, dynamic blocks, Terraform functions, and regular expressions to create flexible and reusable infrastructure configurations

7. Administering Cloud BLOB Storage

Introduction

Object storage is designed to store large volumes of unstructured data, such as media files, documents, backups, and logs. It provides scalability, durability, and cost-effective storage, making it ideal for applications that require efficient handling of massive datasets. Most major cloud providers offer object storage solutions, including AWS S3, Azure Blob Storage, and Google Cloud Storage (GCS). While their implementations vary slightly, they share common concepts such as storage accounts, containers/buckets, objects (blobs), access tiers, and lifecycle policies.

Overview of Object Storage in AWS, Azure, and Google Cloud

Each cloud provider has its own implementation of object storage, but they follow a similar structure:

Concept	AWS S3	Azure Blob Storage	Google Cloud Storage (GCS)
Storage Account	S3 Bucket	Storage Account	GCS Bucket
Container	N/A (S3 uses flat storage)	Storage Container	N/A (GCS uses flat storage)
Object Types	Objects (Standard, Intelligent-Tiering, Glacier)	Blobs (Block, Append, Page)	Objects (Standard, Nearline, Coldline, Archive)
Access Tiers	Standard, IA, Glacier	Hot, Cool, Archive	Standard, Nearline, Coldline, Archive
Replication	Cross-Region, Same-Region	LRS, GRS, RA-GRS	Multi-Region, Dual-Region, Regional

Despite these differences, Terraform provides a consistent way to define and manage object storage across these providers.

Key Features of Cloud Object Storage

All cloud object storage services support:

- Scalability & Durability – Designed to handle petabytes of data with automatic redundancy.
- Access Tiers – Optimize storage costs based on how frequently data is accessed.
    - Frequent Access: AWS S3 Standard, Azure Hot, GCP Standard
    - Infrequent Access: AWS S3 IA, Azure Cool, GCP Nearline
    - Archival Storage: AWS Glacier, Azure Archive, GCP Archive
- Replication – Ensures high availability by copying data across regions.
    - AWS: Cross-Region Replication (CRR) and Same-Region Replication (SRR)
    - Azure: LRS (Local), GRS (Geo-Redundant), RA-GRS (Read-Access GRS)
    - GCP: Multi-Region, Dual-Region, Regional storage classes

Provisioning Object Storage with Terraform

Terraform enables cloud-agnostic infrastructure provisioning. Below are equivalent Terraform configurations for AWS S3, Azure Blob Storage, and GCP Cloud Storage.

AWS S3 Example

```
resource "aws_s3_bucket" "example" {
 bucket = "example-bucket"
 acl    = "private"
}

resource "aws_s3_bucket_versioning" "example" {
```

```
  bucket = aws_s3_bucket.example.id
  versioning_configuration {
    status = "Enabled"
  }
}
```

This configuration provisions an S3 bucket with versioning enabled to track object changes over time.

Azure Blob Storage Example

```
resource "azurerm_storage_account" "example" {
  name = "examplestorageacct"
  resource_group_name = "example-resources"
  location = "East US"
  account_tier = "Standard"
  account_replication_type = "LRS"
}

resource "azurerm_storage_container" "example" {
  name                  = "example-container"
  storage_account_name  = azurerm_storage_account.example.name
  container_access_type = "private"
}
```

Here, we define an Azure Storage Account and a private Blob Storage container.

Google Cloud Storage Example

```
resource "google_storage_bucket" "example" {
  name = "example-bucket"
  location = "US"
  storage_class = "STANDARD"
  versioning {
    enabled = true
  }
}
```

This defines a Google Cloud Storage bucket with versioning enabled.

Managing Object Storage Configurations Across Clouds

All providers offer additional configuration options, including versioning, access policies, and lifecycle management.

Versioning

Versioning helps track changes and prevent accidental deletions.

- **AWS S3**: Enabled using aws_s3_bucket_versioning
- **Azure Blob Storage**: Managed through azurerm_storage_management_policy
- **Google Cloud Storage**: Controlled via versioning { enabled = true } in the bucket configuration

Access Policies and Security

Securing object storage is critical. SAS tokens (Azure), IAM policies (AWS), and Signed URLs (GCP) are commonly used for controlled access.

Example: AWS S3 Bucket Policy for Restricted Access

```
resource "aws_s3_bucket_policy" "example" {
 bucket = aws_s3_bucket.example.id
 policy = <<EOF
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Effect": "Allow",
   "Principal": "*",
   "Action": "s3:GetObject",
   "Resource": "arn:aws:s3:::example-bucket/*"
  }
 ]
}
EOF
}
```

This policy allows public read access to objects in an S3 bucket.

Lifecycle Management

Object storage lifecycle policies help optimize costs by automatically transitioning objects between storage classes or deleting them after a set period.

AWS S3 Lifecycle Rule: Move Objects to Glacier After 30 Days

```
resource "aws_s3_bucket_lifecycle_configuration" "example" {
 bucket = aws_s3_bucket.example.id

 rule {
  id     = "move-to-glacier"
  status = "Enabled"

  transition {
   days          = 30
   storage_class = "GLACIER"
  }
 }
}
```

Azure Blob Storage Lifecycle Policy: Delete Blobs After 30 Days

```
resource "azurerm_storage_management_policy" "example" {
 storage_account_id = azurerm_storage_account.example.id

 rule {
  name    = "expire-old-blobs"
  enabled = true

  filters {
   blob_types = ["blockBlob"]
  }

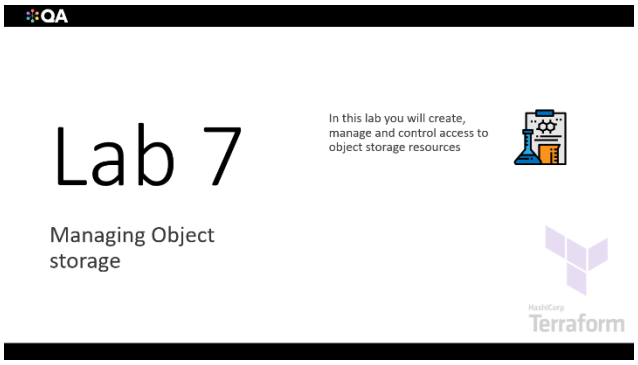
  actions {
   base_blob {
    delete_after_days_since_modification_greater_than = 30
   }
  }
 }
}
```

GCP Cloud Storage Lifecycle Rule: Delete Objects After 30 Days

```
resource "google_storage_bucket" "example" {
 name          = "example-bucket"
 location      = "US"
 storage_class = "STANDARD"
```

```
  lifecycle_rule {
    action {
      type = "Delete"
    }
    condition {
      age = 30
    }
  }
}
```

Lab 7 Managing Object Storage



The lab will guide you through provisioning and managing cloud object storage.

8. Terraform Pipelining

Terraform can be used through its command-line tool or with a shared platform for team workflows.

Terraform Community Edition

Terraform Community Edition is the downloadable command-line tool used throughout this course. It runs on a workstation or a CI/CD runner and manages configuration, providers, infrastructure and state. Current releases are source-available under the Business Source License 1.1. Use Community Edition as the product name when distinguishing it from the managed platforms. See HashiCorp's published licence terms for permitted use.

Terraform Enterprise

Terraform Enterprise is the self-hosted distribution of HCP Terraform. It provides a private instance of the shared management application, operated within infrastructure controlled by the organisation. The organisation is responsible for deploying and maintaining that application. Capabilities depend on the installed version and licence; Enterprise is not simply a different Terraform CLI.

HCP Terraform

HCP Terraform is HashiCorp's hosted service for managing Terraform as a team. It was formerly called Terraform Cloud; the name changed on 22 April 2024. It brings together shared state, managed runs, access controls and integrations. Free and paid editions are available; feature access and limits depend on the edition.

Remote State Management

HCP Terraform keeps versioned state for each workspace in a shared service. Team members and automation use the workspace's state instead of exchanging local files. Access controls and run coordination help protect it from unauthorised or conflicting changes. State can contain sensitive data, so permissions remain important.

Collaboration and Teamwork

Teams can review plans, follow runs and control who may act on infrastructure. Access is organised through the service's organisation, project and workspace permissions, subject to the edition in use. This makes responsibility visible; it does not remove the need for agreed ownership and approval practices.

Version Control Integration

A workspace can be connected to a supported version control system such as GitHub, GitLab, Bitbucket or Azure DevOps. Repository changes can trigger plans, and pull requests can receive speculative plans for review. An apply follows the workspace's configured workflow and approval settings. Reverting code requires a new reviewed change; a Git revert does not itself reverse deployed infrastructure.

Policy Enforcement and Governance

HCP Terraform can evaluate policy checks within managed run workflows. Supported frameworks include Sentinel and Open Policy Agent (OPA). Policy sets, permissions and approval settings can help enforce organisational requirements. Their availability and behaviour depend on the edition, execution mode and configuration; a remote state store alone is not a complete governance process.

Workspaces and Environment Management

A workspace manages one collection of infrastructure through its configuration, state, variables and run settings. Separate development, test and production workspaces are one possible arrangement. A workspace can instead represent a component such as networking. Projects group related workspaces and can provide an access boundary. Choose boundaries to reflect ownership and lifecycle.

Execution Modes and Runs

Remote execution runs Terraform on HashiCorp-managed workers. Agent execution uses an agent in infrastructure controlled by your organisation. Local execution runs Terraform on your workstation or CI/CD runner while HCP Terraform stores the state. Many managed-run features are unavailable in local mode; workspace variables and variable sets are not evaluated there.

Integration with Team Workflows

HCP Terraform supports VCS-driven, CLI-driven and API-driven workflows. An external pipeline can initiate or monitor a managed run, or it can run Terraform itself and use HCP Terraform for state storage. These are different operating models. Decide where execution, plan review and approval belong, and avoid having two independent systems apply changes to the same state.

For current product guidance, examples and feature information, refer to the HCP Terraform documentation.

HCP Terraform Workspaces

An HCP Terraform workspace is a managed context for one collection of infrastructure. It brings together the configuration source, state history, variables and execution settings. When HCP Terraform manages runs, it also records run activity. It is more than a name for an environment or an alternative folder.

HCP Workspaces and CLI Workspaces

Terraform CLI workspaces select separate state instances within the same working directory and configuration. HCP Terraform workspaces are separate managed contexts, each with its own configuration source, variables, state and settings. The two features share a name but are not interchangeable. Creating a CLI workspace does not by itself establish an HCP project, permissions or a governed deployment workflow.

Projects and Environment Separation

Projects organise related workspaces, for example by application, team or business unit. Project-level access controls depend on the edition. A project can contain separate networking-dev, networking-test and networking-prod workspaces. Their state is separate, but promotion remains an explicit decision about which configuration or module version each workspace consumes.

Variable Management

For managed runs, use Terraform variables for declared inputs and environment variables for runner or provider settings. Variable sets can share values across several workspaces. Sensitive values should be handled through the appropriate security

controls. In local execution mode, supply values through the local runner; HCP workspace variables and variable sets are not evaluated.

State Management

Each workspace retains its own state versions. Treat access to state as access to potentially sensitive infrastructure information. Keeping state separate does not itself isolate cloud resources: provider credentials, subscriptions or accounts, network boundaries and permissions must also be designed appropriately.

Collaboration and Access Control

Grant the permissions needed to view plans, manage variables and approve or execute changes. HCP Terraform permissions control access to the service; cloud-provider permissions control what a run can do to infrastructure. Both boundaries matter. Project and workspace access should follow the organisation's ownership model.

Workspace Configuration and Module Reuse

A workspace runs a root Terraform configuration obtained from a repository or supplied through the CLI or API. That configuration calls modules using source and version references. Modules are reusable code consumed by the configuration; they are not a separate set of resources automatically created by the workspace. Use deliberate version selection when promoting changes.

Provider Credentials

Where supported, dynamic provider credentials use workload identity and OIDC trust to obtain temporary access for a run. This can reduce reliance on long-lived cloud secrets stored in variables. Authentication still needs an explicit trust relationship and appropriately scoped cloud permissions.

Run History and Logging

For runs managed by HCP Terraform, workspace history records plans, applies, logs, outcomes and associated changes. Use it to review what happened and support troubleshooting. With local execution, the external runner produces the execution logs; storing state in HCP Terraform does not automatically capture every local command.

How This Relates to the Pipeline Lab

The Jenkins lab demonstrates Terraform automation through an external CI/CD tool. It does not require HCP Terraform. The same core workflow can be automated using other pipeline platforms. HCP Terraform is an additional way to organise state, execution and governance; choose a model that makes responsibility and approval clear.

Terraform in a pipeline

A Terraform pipeline runs the commands you already know on a controlled execution machine. The pipeline platform decides when work starts, which identity it uses, where results are recorded and whether a change may proceed. Terraform still calculates and applies the infrastructure changes.

The Jenkins lab is one implementation of this pattern. GitHub Actions and Azure Pipelines provide alternative implementations. Their terminology and configuration differ, but the same separation between validation, planning, review and execution applies. These sections introduce the options; the existing Jenkins lab remains the worked exercise.

Step	What happens	Evidence or control
Check the source	Check out a specific commit and run formatting checks.	Known source revision and consistent style.
Initialise and validate	Install the agreed Terraform version, initialise providers/backend and validate configuration.	Expected providers, backend target and valid configuration.
Create a plan	Authenticate, read the target state and calculate changes.	Readable plan and, for an apply workflow, a saved plan file.
Review and approve	Review the proposed changes before allowing the apply job to start.	Approval linked to the intended environment and plan.
Apply	Use the saved plan from the approved deployment run.	Run result and updated remote state.
Record the outcome	Retain appropriate logs and deployment information.	Traceable source, target, approval and result.

Pull request plans and deployment plans

A pull request plan supports code review. After merge, produce the deployment plan from the exact approved revision, review that plan and gate its apply. A plan generated before merge is not automatically the correct artefact for deployment. If the configuration or state changes and the plan becomes invalid, create a new plan and repeat the review.

A shared Terraform command sequence

After the runner is prepared and authenticated, the planning job can run the commands below from the intended environment root. The backend and dev.tfvars must already be configured for that environment.

```
terraform fmt -check -recursive
terraform init -input=false
terraform validate
```

```
terraform plan -input=false -var-file=dev.tfvars -out=tfplan
terraform show -no-color tfplan
```

A later apply job restores the same source revision, Terraform version, provider lock file and saved plan, initialises the same backend, obtains fresh credentials, and runs terraform apply -input=false tfplan. A saved-plan apply does not ask for another interactive confirmation, so the pipeline approval must happen before that command.

Saved plans can contain sensitive data. Restrict artefact access and retention. Use remote state with locking and coordinate pipelines that target the same state; an approval gate does not make concurrent changes safe.

GitHub Actions

GitHub Actions runs automation associated with a GitHub repository. Terraform configuration and workflow definitions can be reviewed together, and runs can start from pull requests, branch updates or a manual request. The workflow is written in YAML and stored under .github/workflows/.

Core components

Component	Role in a Terraform workflow
Workflow	The YAML definition of triggers, permissions and jobs.
Job	A group of steps, such as validation, planning or applying.
Runner	The hosted or self-hosted machine that executes a job.
Step or action	A script command or reusable component, such as checkout, authentication or artefact transfer.
Environment	A named deployment target with variables, secrets and applicable protection rules.
Artefact	A retained file, such as a saved deployment plan or plan summary.

An example Terraform workflow

A developer opens a pull request. A validation job checks formatting and configuration; a separate plan job may create a preview using an appropriately limited identity. After the reviewed change is merged, a deployment run creates a saved plan for the target environment. The apply job depends on that planning job, references a protected environment, restores the matching artefact and applies it only after the required checks pass.

Authentication and state access

OpenID Connect (OIDC) can exchange the workflow identity for temporary cloud access. The job needs id-token: write to request a token, and the cloud must trust the intended repository, branch or environment. This permission does not itself grant

access to cloud resources. Backend access and provider permissions must both be configured. Keep cloud credentials away from untrusted pull request code.

Approvals and environment protections

Referencing an environment does not create an approval rule. Configure required reviewers and branch restrictions on the environment where available. Availability depends on the GitHub plan and repository visibility, especially for private repositories. A workflow_dispatch trigger starts a manual run; it is not, by itself, a separate deployment approval.

Azure DevOps Pipelines

Azure Pipelines is the pipeline service within Azure DevOps. It can run Terraform for Azure and other platforms when the required tools and credentials are available. A YAML pipeline is usually stored as azure-pipelines.yml. Source can be held in Azure Repos or another supported repository, including GitHub.

Core components

Component	Role in a Terraform pipeline
Pipeline and stage	The overall workflow and its logical phases, such as Validate, Plan and Apply.
Job and agent	A unit of work and the Microsoft-hosted or self-hosted machine that executes it.
Task or script step	A reusable task or command that installs tools, authenticates or runs Terraform.
Service connection	An authorised connection to an external service, including an Azure identity.
Environment	A deployment target that can have approvals and checks.
Pipeline artefact	A retained output transferred between stages or jobs, such as a saved plan.

An example Terraform pipeline

A validation stage checks the proposed change. A planning stage authenticates through the configured connection, initialises the target backend and publishes the saved plan and its readable summary. An Apply stage depends on successful planning. A deployment job references the target environment; after its approval checks pass, the job retrieves the same plan and applies it.

Authentication through service connections

For Azure deployments, use an Azure Resource Manager service connection with workload identity federation where supported. Authorise only the pipelines that need it

and assign the underlying identity appropriate Azure permissions. Configure the Terraform task or script for federated authentication to both the backend and provider; a successful Azure login alone does not prove that every Terraform authentication path is configured.

Approvals and repository checks

Resource owners configure approvals and checks on resources such as environments and service connections. These controls are managed outside the pipeline YAML. Merely naming an environment in a deployment job does not create a review requirement. For Azure Repos Git, configure pull request build validation through the target branch policy; do not assume a YAML pr trigger provides it.

Comparing the pipeline options

The platforms provide different ways to organise the same responsibilities. Select an approach that fits the organisation's repositories, identity model, operating skills and required controls. Using a platform already supported by the organisation can reduce the amount of new infrastructure and administration needed.

Responsibility	Jenkins	GitHub Actions	Azure Pipelines
Workflow definition	Jenkinsfile	YAML workflow in .github/workflows/	YAML pipeline, often azure-pipelines.yml
Execution machine	Jenkins agent	GitHub-hosted or self-hosted runner	Microsoft-hosted or self-hosted agent
Reusable steps	Pipeline steps and plugins	Actions and reusable workflows	Tasks and YAML templates
Review gate	Pipeline approval step and related permissions	Environment protections, where available	Resource approvals and checks
Cloud access	Configured credentials or supported federation	Cloud trust using OIDC, where supported	Service connection or configured credentials; Azure federation where supported
Terraform execution	CLI on the agent	CLI on the runner	CLI or supported tasks on the agent

Where HCP Terraform fits

Jenkins, GitHub Actions and Azure Pipelines can run Terraform themselves, or they can integrate with HCP Terraform and initiate managed runs. In the first model, design state storage, identity, approvals and concurrency in the external pipeline. In the second, decide which responsibilities remain in the pipeline, and which are enforced by HCP Terraform. A single state must have one coordinated change process.

9. Bonus Module: Administering Cloud RDBMS

Introduction

Relational databases (RDBMS) store structured data using tables with relationships defined between them. They support SQL-based queries for efficient data retrieval, manipulation, and transaction handling. Cloud providers offer fully managed RDBMS services, which automate database provisioning, backups, scaling, and security, reducing administrative overhead. The most widely used cloud-based relational database services include AWS RDS, Azure SQL Database, and Google Cloud SQL, each providing built-in high availability, scalability, and advanced security features.

Overview of RDBMS Offerings in AWS, Azure, and Google Cloud

Each cloud provider offers managed relational database services with similar capabilities:

Feature	AWS RDS	Azure SQL Database	Google Cloud SQL
Database Engine Support	MySQL, PostgreSQL, SQL Server, MariaDB, Oracle	SQL Server (only)	MySQL, PostgreSQL, SQL Server
High Availability	Multi-AZ Deployments	Geo-Replication, Zone Redundancy	High-Availability Configuration
Scalability	Read Replicas, Auto Scaling	Elastic Pools, Serverless	Read Replicas, Auto Scaling
Security Features	IAM Authentication, Encryption, Security Groups	Threat Detection, Encryption, Firewall Rules	IAM Authentication, Encryption, Private IP

Despite variations, Terraform provides a consistent way to deploy and manage relational databases across these platforms.

Key Features of Cloud RDBMS Solutions

Managed relational databases in AWS, Azure, and GCP provide:

- High Availability & Redundancy – Ensuring minimal downtime through multi-zone replication.
- Scalability – Options for vertical scaling (upgrading instance sizes) and horizontal scaling (read replicas, sharding).
- Security & Compliance – Features like encryption at rest & in transit, IAM-based authentication, and firewall rules to restrict access.
- Automated Backups – Scheduled and on-demand backups for disaster recovery.

Provisioning Relational Databases with Terraform

Terraform allows cloud-agnostic provisioning of relational databases. Below are equivalent configurations for AWS RDS, Azure SQL Database, and GCP Cloud SQL.

AWS RDS Example

```
resource "aws_db_instance" "example" {
 identifier = "example-db"
 engine = "mysql"
 instance_class = "db.t3.micro"
 allocated_storage = 20
 username = "admin"
 password = "P@ssword1234!"
 publicly_accessible   = false
 skip_final_snapshot    = true
}
```

This provisions an Amazon RDS MySQL database with private access.

Azure SQL Database Example

```
resource "azurerm_sql_server" "example" {
 name = "example-sqlserver"
 resource_group_name = var.resource_group_name
 location = var.location
 version = "12.0"
 administrator_login = "sqladmin"
 administrator_login_password = "P@ssword1234!"
}

resource "azurerm_sql_database" "example" {
 name = "example-db"
 resource_group_name = var.resource_group_name
 location = var.location
 server_name = azurerm_sql_server.example.name
 sku_name = "Basic"
}
```

This deploys an Azure SQL Database under an SQL Server instance.

Google Cloud SQL Example

```
resource "google_sql_database_instance" "example" {
 name = "example-db"
 database_version = "MYSQL_8_0"
```

```
 region = "us-central1"
 settings {
  tier = "db-f1-micro"
 }
}

resource "google_sql_database" "example" {
 name = "example-db"
 instance = google_sql_database_instance.example.name
}
```

This Terraform configuration sets up a Cloud SQL MySQL database in Google Cloud.

Database Configuration and Management

All cloud providers offer configurations to enhance database security, access control, and performance optimization.

Firewall Rules and Network Security

Restricting database access is essential for security. AWS, Azure, and GCP allow firewall-based access control.

AWS RDS Security Group Rule (Restrict to Specific IPs)

```
resource "aws_security_group_rule" "allow_db_access" {
 type = "ingress"
 from_port = 3306
 to_port = 3306
 protocol = "tcp"
 cidr_blocks = ["192.168.1.0/24"]
}
```

Azure SQL Firewall Rule (Allowing Specific IP Ranges)

```
resource "azurerm_sql_firewall_rule" "example" {
 name = "AllowMyIP"
 resource_group_name = var.resource_group_name
 server_name = azurerm_sql_server.example.name
 start_ip_address = "0.0.0.0"
 end_ip_address = "255.255.255.255"
}
```

GCP Cloud SQL Authorized Networks (Restricting Access to an IP Range)

```
resource "google_sql_user" "example" {
 name    = "admin"
 instance = google_sql_database_instance.example.name
```

```
  password = "P@ssword1234!"
}

resource "google_sql_database_instance" "example" {
 settings {
  ip_configuration {
   authorized_networks {
    value = "192.168.1.0/24"
   }
  }
 }
}
```

These configurations ensure that only authorized networks can access the database.

Monitoring & Auditing

Database auditing helps track user activity and detect security threats.

AWS RDS Enhanced Monitoring

```
resource "aws_db_instance" "example" {
 monitoring_interval = 60
}
```

Azure SQL Database Auditing Policy

```
resource "azurerm_sql_database_extended_auditing_policy" "example" {
 database_id = azurerm_sql_database.example.id
 storage_endpoint  =
azurerm_storage_account.example.primary_blob_endpoint
 retention_in_days = 30
}
```

GCP Cloud SQL Logging & Monitoring

```
resource "google_logging_project_sink" "example" {
 name = "example-sql-logs"
 destination =
"storage.googleapis.com/${google_storage_bucket.example.name}"
}
```

These configurations help track database performance, detect anomalies, and meet compliance requirements.

10. Bonus Module - Kubernetes Management with Terraform

Introduction



Kubernetes is an open-source platform for orchestrating and managing containerized applications. It simplifies deployment, scaling, and operations by distributing workloads across multiple nodes, ensuring high availability and fault tolerance. Kubernetes is widely adopted across cloud providers, including Azure Kubernetes Service (AKS), Amazon Elastic Kubernetes Service (EKS), and Google Kubernetes Engine (GKE), each offering managed solutions with built-in integrations and security features.

Key Components of Kubernetes

Kubernetes is built on several core components that work together to manage containerized applications:

Clusters – Groups of nodes managed together as a single unit.

Nodes – Physical or virtual machines that run Kubernetes workloads.

Deployments & Services – Kubernetes objects used to manage application scaling and networking.

Pods – The smallest deployable unit in Kubernetes, consisting of one or more containers.

Most cloud providers offer managed Kubernetes services to reduce operational complexity and automate infrastructure management.

Overview of Managed Kubernetes Services (AKS, EKS, GKE)

Each cloud provider offers a managed Kubernetes service, providing similar functionality with slight variations:

Factor	AKS	EKS	GKE
Control Plane Cost	Free	Paid ($0.10/hr)	Free for Standard mode
Node Autoscaling	VM Scale Sets	Cluster Autoscaler	Built-in with Autopilot
Load Balancing	Azure Load Balancer	AWS Load Balancer Controller	GCP Network Load Balancer
Storage Options	Azure Disks, Azure Files	EBS, EFS, S3	Persistent Disk, Filestore
IAM & Authentication	Azure AD, RBAC	IAM Roles for Service Accounts	Workload Identity
Monitoring & Logging	**Azure Monitor**	**AWS CloudWatch & Prometheus**	**Google Cloud Observability** (Cloud Monitoring & Logging)

Despite these differences, Kubernetes clusters in AKS, EKS, and GKE can be provisioned using Terraform for a consistent infrastructure-as-code approach.

Provisioning Kubernetes Clusters with Terraform

Terraform enables multi-cloud Kubernetes deployments by providing consistent, repeatable infrastructure configurations.

Deploying an Azure Kubernetes Service (AKS) Cluster

```
resource "azurerm_kubernetes_cluster" "example" {
 name           = "example-aks"
 location       = var.location
 resource_group_name = var.resource_group_name
 dns_prefix     = "exampleaks"

 default_node_pool {
  name      = "default"
  node_count = 2
  vm_size    = "Standard_DS2_v2"
 }

 identity {
  type = "SystemAssigned"
 }
}
```

This configuration creates a 2-node AKS cluster with a system-assigned identity.

Deploying an AWS Elastic Kubernetes Service (EKS) Cluster

```
resource "aws_eks_cluster" "example" {
```

```
  name     = "example-eks"
  role_arn = aws_iam_role.eks.arn

  vpc_config {
   subnet_ids = aws_subnet.eks[*].id
  }
}

resource "aws_eks_node_group" "example" {
  cluster_name  = aws_eks_cluster.example.name
  node_role_arn = aws_iam_role.eks_nodes.arn
  subnet_ids    = aws_subnet.eks[*].id
  scaling_config {
   desired_size = 2
   min_size     = 1
   max_size     = 3
  }
}
```

This configuration provisions an EKS cluster with a node group supporting autoscaling.

Deploying a Google Kubernetes Engine (GKE) Cluster

```
resource "google_container_cluster" "example" {
  name     = "example-gke"
  location = "us-central1"
  initial_node_count = 2

  node_config {
   machine_type = "e2-medium"
  }
}
```

This configuration creates a 2-node GKE cluster in the us-central1 region.

Managing Kubernetes Resources with Terraform

Once a cluster is deployed, Terraform's Kubernetes provider can be used to manage workloads within the cluster.

Example: Deploying a Nginx Application in Kubernetes

The following example defines a Nginx deployment and exposes it via a service.

Provider Configuration (For AKS)

```
provider "kubernetes" {
 host = azurerm_kubernetes_cluster.example.kube_config[0].host
 client_certificate =
base64decode(azurerm_kubernetes_cluster.example.kube_config[0].client_
certificate)
 client_key =
base64decode(azurerm_kubernetes_cluster.example.kube_config[0].client_
key)
 cluster_ca_certificate =
base64decode(azurerm_kubernetes_cluster.example.kube_config[0].cluste
r_ca_certificate)
}
```

Kubernetes Deployment

```
resource "kubernetes_deployment" "nginx" {
 metadata {
  name = "nginx-deployment"
 }
 spec {
  replicas = 2
  selector {
   match_labels = {
    app = "nginx"
   }
  }
  template {
   metadata {
    labels = {
     app = "nginx"
    }
   }
   spec {
    container {
     name  = "nginx"
     image = "nginx:1.14.2"
     ports {
      container_port = 80
     }
    }
   }
  }
 }
}
```

Kubernetes Service to Expose the Deployment

```
resource "kubernetes_service" "nginx" {
 metadata {
  name = "nginx-service"
 }
 spec {
  selector = {
   app = "nginx"
  }
  port {
   port       = 80
   target_port = 80
  }
  type = "LoadBalancer"
 }
}
```

This configuration:

- Deploys two Nginx pods within the cluster.
- Creates a Kubernetes service to expose the application externally.

Multi-Cloud Kubernetes Deployment Considerations

Deploying Kubernetes clusters across AWS, Azure, and GCP requires consideration of platform-specific differences:

Factor	AKS	EKS	GKE
Control Plane Cost	Free	Paid ($0.10/hr)	Free for Standard mode
Node Autoscaling	VM Scale Sets	Cluster Autoscaler	Built-in with Autopilot
Load Balancing	Azure Load Balancer	AWS Load Balancer Controller	GCP Network Load Balancer
Storage Options	Azure Disks, Azure Files	EBS, EFS, S3	Persistent Disk, Filestore
IAM & Authentication	Azure AD, RBAC	IAM Roles for Service Accounts	Workload Identity

Terraform and Kubernetes State Management

As you know, Terraform manages infrastructure declaratively using a **state file** to track resources it creates. However, Kubernetes operates dynamically, adjusting workloads and resources based on system conditions. This **difference in state management** can cause conflicts when Terraform attempts to enforce a desired state.

Common Issues with Terraform Managing Kubernetes Resources:

- **Autoscaling Conflicts:** If Kubernetes scales pods dynamically (e.g., Horizontal Pod Autoscaler), Terraform's state may become outdated. Running terraform apply can unintentionally revert Kubernetes' scaling decisions.
- **Drift Between Terraform and Kubernetes:** Kubernetes can recreate or reschedule pods for various reasons (node failures, resource optimizations). Terraform does not track these changes unless explicitly refreshed.
- **Limited Control over Kubernetes-Specific Features:** Terraform manages resources at the infrastructure level, while Kubernetes handles workloads at the application level. This separation can make managing Kubernetes-native features more cumbersome with Terraform alone.

To avoid these issues, it is often beneficial to use **Helm** to manage Kubernetes workloads while Terraform provisions the infrastructure.

Introducing Helm charts



Helm is a powerful package manager for Kubernetes that simplifies the deployment, configuration, and management of applications within a Kubernetes cluster. Much like how apt or yum manage packages on Linux systems, Helm manages Kubernetes applications through reusable templates known as **Helm charts**. These charts bundle Kubernetes manifests (like deployments, services, and config maps) with default configuration values, allowing users to deploy complex applications with just a few commands or configuration adjustments.

When managing Kubernetes with Terraform, Helm plays a crucial role in reducing complexity. While Terraform excels at infrastructure provisioning—such as setting up clusters, networking, and cloud resources—managing dynamic, application-level resources like deployments, services, and autoscalers can become cumbersome. Defining these resources directly with Terraform requires extensive boilerplate code, and handling updates, rollbacks, or versioning manually increases operational overhead.

Integrating Helm with Terraform bridges this gap. Terraform's **Helm provider** allows you to deploy and manage Helm charts declaratively, just like other infrastructure components. Instead of writing verbose Kubernetes resource definitions, you simply reference a Helm chart and specify key configuration values. This approach reduces code duplication, simplifies resource management, and leverages the Kubernetes community's best practices baked into popular charts. Additionally, Helm's versioning

and rollback features provide greater flexibility in managing application lifecycle changes, making deployments more resilient and adaptable in dynamic environments.

By combining Terraform's infrastructure-as-code capabilities with Helm's application management strengths, teams can achieve a more streamlined, efficient, and scalable approach to Kubernetes operations.

Appendix. Terraform Associate 004 course coverage

QATIP QATIPINT and QATFADV

The QATIP, QATIPINT and QATFADV courses were developed to build practical capability in Terraform, rather than as dedicated exam preparation courses. Their structure and content follow a progressive learning journey, introducing Terraform and then exploring its use in increasing depth within organisations.

The journey moves from understanding the core concepts and deploying infrastructure to designing reusable configurations, managing state and environments, and controlling change safely. As learners progress, the emphasis extends to collaboration, ownership, governance and the judgement needed to operate Terraform in shared, long-lived environments.

This appendix provides an additional reference showing how that learning journey aligns with the Terraform Associate 004 exam objectives. It identifies relevant coverage across the three courses and areas where further study or practice may be needed. The mapping supports certification planning; it does not redefine the courses as exam preparation or imply that completing them alone establishes exam readiness.

Reading the matrix

Covered Explicit teaching or examples address the objective.

Partial Only part is addressed, or a practical objective has conceptual coverage only.

Not evidenced No explicit treatment was located in the manual; separate labs or trainer explanations may add coverage.

ID	Objective outline	QATIP	QATIPINT	QATFADV
1 Infrastructure as code				
1a	Meaning of IaC	**Covered** Sec. 1, pp. 3-4: code-defined infrastructure.	**Covered** Sec. 1, p. 4: refresher.	**Partial** Mod. 0, pp. 6-7: operating context; basics assumed.
1b	Why codify infrastructure	**Covered** Sec. 1, pp. 3-5: repeatability and collaboration.	**Covered** Sec. 1, p. 4: consistency and repeatability.	**Covered** Mod. 0, pp. 6-7: safe and deliberate change.
1c	Cross-platform workflows	**Covered** Sec. 1, pp. 4-7; Sec. 3, pp. 32-37: provider independence.	**Covered** Secs. 1, 3 and 7, pp. 4, 8-16, 31-35: cloud comparisons.	**Not evidenced** Azure-specific scenario; no cross-platform explanation.

ID	Objective outline	QATIP	QATIPINT	QATFADV
2 Terraform fundamentals				
2a	Provider installation and versions	**Partial** Secs. 1-3, pp. 9, 15, 23, 33: installation and version example; lock-file workflow not developed.	**Partial** Sec. 1, p. 5: init downloads providers; version controls not developed.	**Not evidenced** Provider installation and version controls are assumed.
2b	Provider responsibilities	**Covered** Sec. 3, pp. 32-35: plugins, resources and data sources.	**Partial** Sec. 1, pp. 4-5: provider role introduced briefly.	**Not evidenced** Azure resources used; provider architecture not explained.
2c	Combining providers	**Covered** Sec. 3, pp. 33-34: local-file data passed to AWS resources.	**Partial** Bonus Sec. 10, pp. 50-53: cloud and Kubernetes providers; bonus content only.	**Not evidenced** No explicit combined-provider example.
2d	State purpose and use	**Covered** Sec. 5, pp. 49-57: identity, storage, drift and import.	**Partial** Sec. 8, pp. 36-39: hosted state; Bonus Sec. 10, pp. 53-54: drift.	**Covered** Mods. 1, 4 and 8, pp. 7-13, 22-27, 40-45: identity and ownership.

Workflow coverage

ID	Objective outline	QATIP	QATIPINT	QATFADV
3 Core workflow				
3a	Workflow stages	**Covered** Secs. 1-2, pp. 7-8, 15-27: write through deployment.	**Covered** Sec. 1, p. 5; Sec. 8, pp. 41-44: workflow and automation.	**Partial** Mods. 0 and 6-8, pp. 6-7, 32-45: operating workflow; basics assumed.
3b	Working-directory initialisation	**Covered** Sec. 2, p. 23: init and backend setup.	**Covered** Secs. 1 and 8, pp. 5, 41-44: init and pipeline stage.	**Covered** Mods. 4 and 8, pp. 22-27, 42: backend reconfiguration.
3c	Configuration validation command	**Partial** Sec. 6, pp. 66-68: validation discussed; validate command not demonstrated.	**Partial** Secs. 5 and 8, pp. 21-25, 39-40: checks discussed; validate command not demonstrated.	**Partial** Mods. 2 and 5, pp. 13-19, 27-31: custom checks; validate command not demonstrated.

ID	Objective outline	QATIP	QATIPINT	QATFADV
3d	Plan generation and review	**Covered** Sec. 2, pp. 24-25: plans and saved-plan use.	**Covered** Secs. 1 and 8, pp. 5, 41-44: plan stage and review.	**Covered** Mods. 5-8, pp. 30-45: zero-diff refactoring, promotion and risk review.
3e	Applying planned changes	**Covered** Sec. 2, pp. 25-27: apply workflow.	**Covered** Secs. 1 and 8, pp. 5, 41-44: apply and approval gate.	**Partial** Mods. 6-8, pp. 32-45: governed execution; basic CLI use assumed.
3f	Removing managed infrastructure	**Covered** Secs. 1-2, pp. 18, 27: destroy and cleanup.	**Covered** Sec. 1, p. 5: destroy stage and lab cleanup.	**Partial** Mods. 1 and 8, pp. 8-13, 40-45: destructive impact; destroy CLI not taught.
3g	Consistent formatting	**Covered** Sec. 2, pp. 21-22: fmt.	**Not evidenced** No explicit fmt treatment located.	**Not evidenced** No explicit fmt treatment located.

Preparation focus

Practise the complete CLI cycle, including terraform validate and fmt. Custom input checks do not replace the validate command. QATFADV assumes basic CLI operation and concentrates on change impact and execution control. Coverage is an editorial assessment of the manuals, not HashiCorp endorsement or a guarantee of passing.

Configuration coverage

ID	Objective outline	QATIP	QATIPINT	QATFADV
4 Configuration authoring				
4a	Managed resources versus lookups	**Covered** Secs. 1 and 3, pp. 10-13, 32-34: resource and data examples.	**Partial** Secs. 3 and 7, pp. 11-12, 31-35: examples; distinction not fully explained.	**Covered** Mod. 8, pp. 41-45: import ownership versus platform data sources.
4b	Attribute links between resources	**Covered** Secs. 1-2 and 4, pp. 11-13, 29-30, 45: named references.	**Covered** Sec. 4, pp. 17-20; Sec. 7, pp. 32-35: resource and module references.	**Covered** Mods. 3, 5 and 8, pp. 19-22, 28-30, 42-45: wiring and lookups.
4c	Inputs and exported values	**Covered** Sec. 4, pp. 37-41: variables, precedence and outputs.	**Covered** Secs. 2 and 4, pp. 7-8, 17-20: inputs and module outputs.	**Covered** Mods. 2 and 5-6, pp. 13-19, 27-33: inputs and contracts.

ID	Objective outline	QATIP	QATIPINT	QATFADV
4d	Structured and collection values	**Covered** Sec. 4, pp. 45-47: list, map, object, tuple and set.	**Partial** Secs. 2 and 6, pp. 7-8, 25-28: arrays/maps and iteration; type coverage limited.	**Covered** Mods. 1-3, pp. 9-11, 13-22: collections and normalised input structures.
4e	Expressions and dynamic generation	**Covered** Sec. 4, pp. 41-47: functions, conditions and iteration.	**Covered** Secs. 4 and 6, pp. 19-20, 25-30: templates, functions and dynamic blocks.	**Covered** Mods. 1-3, pp. 7-22: for_each, locals, expressions and dynamic configuration.
4f	Dependencies and lifecycle ordering	**Partial** Sec. 2, pp. 29-31: implicit/explicit dependencies; create_before_destroy not evidenced.	**Partial** Secs. 4 and 7, pp. 17-20, 32-35: reference-based links; explicit/lifecycle ordering not developed.	**Partial** Mods. 3 and 8, pp. 19-22, 43-45: associations and migration; lifecycle syntax not demonstrated.
4g	Custom validation conditions	**Not evidenced** No explicit custom-condition instruction located.	**Covered** Sec. 5, pp. 21-25: variable validation, preconditions and postconditions.	**Covered** Mods. 2 and 5, pp. 16-18, 29: validation and module guardrails.
4h	Sensitive values and Vault	**Partial** Secs. 2 and 5, pp. 19-21, 50-52: credentials/state security; newer features and Vault absent.	**Partial** Sec. 3, pp. 8-16: cloud identity and secret stores; Terraform sensitivity features and Vault absent.	**Partial** Mods. 4 and 7, pp. 22-27, 37-39: access and identity; sensitivity features and Vault absent.

Preparation focus

Complete 4f with explicit dependency and lifecycle-ordering exercises, including create_before_destroy. For 4h, add sensitive variables and outputs, protection of plans and state, Vault, ephemeral values and write-only arguments. Cloud authentication alone does not cover this objective. See official sources 2 and 3.

Modules and state coverage

ID	Objective outline	QATIP	QATIPINT	QATFADV
5 Modules				
5a	Module acquisition	**Partial** Sec. 5, pp. 58-60: local structure and registry; source syntax limited.	**Covered** Sec. 4, pp. 17-20: local, Git and registry sources.	**Partial** Mods. 5-6, pp. 27-33: reuse and publishing; acquisition syntax limited.
5b	Module input scope	**Covered** Sec. 5, pp. 58-60: root/child inputs and output boundaries.	**Covered** Sec. 4, pp. 17-18: child inputs and root arguments.	**Covered** Mods. 5-6, pp. 28-33: root responsibility and child contracts.
5c	Module composition	**Covered** Sec. 5, pp. 58-60: root/child composition and interfaces.	**Covered** Sec. 4, pp. 17-20: network module and root call.	**Covered** Mod. 5, pp. 27-31: refactor and wire security/network modules.
5d	Module release selection	**Not evidenced** Reuse described; version-selection instruction not located.	**Partial** Sec. 4, p. 19: version constraints introduced; source-specific pinning needs practice.	**Covered** Mods. 5-7, pp. 31-39: pinned releases, promotion and controlled change.
6 State operations				
6a	Local state storage	**Covered** Sec. 5, pp. 49-50: default file and local limitations.	**Not evidenced** No explicit local-backend instruction located.	**Covered** Mod. 4, pp. 22-24: local-state limits and remote transition.
6b	Concurrent-operation locking	**Covered** Sec. 5, pp. 50-54: locking purpose and backend behaviour.	**Not evidenced** Hosted state discussed; locking not explicitly developed.	**Covered** Mods. 4 and 8, pp. 23, 40: preventing concurrent state changes.
6c	Remote backend configuration	**Covered** Secs. 1, 2 and 5, pp. 9-10, 23, 50-52: backend examples and init.	**Partial** Sec. 8, pp. 36-39, 43: hosted state and backend mention; configuration not developed.	**Covered** Mods. 4 and 8, pp. 22-27, 42: bootstrap, backend targeting and reconfiguration.
6d	Drift and state reconciliation	**Covered** Sec. 5, pp. 51-57: inspect, restore, drift and import.	**Partial** Bonus Sec. 10, pp. 53-54: Kubernetes drift; state operations not developed.	**Covered** Mods. 1, 5 and 8, pp. 7-13, 30-31, 40-45: identity, moves and reconciliation.

Preparation focus

Pair module-source practice with release selection. Registry version constraints and Git ref pinning use different mechanisms. Add provider version constraints, dependency-lock-file behaviour and a controlled upgrade exercise for 2a. For state, practise safe reconciliation using the current CLI and review every resulting plan.

Maintenance and HCP coverage

ID	Objective outline	QATIP	QATIPINT	QATFADV
7 Infrastructure maintenance				
7a	Adopting existing resources	**Covered** Sec. 5, pp. 55-57: import and configuration alignment.	**Not evidenced** No explicit import treatment located.	**Covered** Mod. 8, pp. 40-45: import, ownership and generated configuration.
7b	Command-line state inspection	**Covered** Sec. 5, pp. 51-52: state list and state show.	**Not evidenced** No explicit state inspection commands located.	**Partial** Mods. 4-5 and 8, pp. 25-26, 30-31, 40-42: metadata/moves; list/show not developed.
7c	Diagnostic logging	**Partial** Sec. 5, pp. 60-62: TF_LOG and TF_LOG_PATH; see correction below.	**Not evidenced** Hosted run history is discussed, but verbose CLI logging is not.	**Not evidenced** No explicit diagnostic-logging instruction located.
8 HCP Terraform				
8a	Hosted infrastructure deployment	**Partial** Sec. 6, pp. 64-66: remote runs described; hosted deployment not demonstrated.	**Partial** Sec. 8, pp. 36-39: remote runs described; hosted deployment not demonstrated.	**Not evidenced** Generic pipelines and remote backends do not establish HCP usage.
8b	Hosted collaboration and controls	**Covered** Sec. 6, pp. 64-66: teams, access and policy features described.	**Covered** Secs. 5 and 8, pp. 23-24, 36-39: hosted policies and team access described.	**Not evidenced** Mods. 6-7 cover governance principles, not HCP-specific features.
8c	Hosted workspaces and projects	**Partial** Sec. 6, pp. 65-66: workspaces; HCP projects not evidenced.	**Partial** Sec. 8, pp. 37-39: workspaces; HCP projects not evidenced.	**Not evidenced** Environment roots/backends are not HCP workspaces or projects.
8d	HCP integration setup and use	**Partial** Sec. 6, pp. 64-66: integrations described; CLI/cloud integration not configured.	**Partial** Sec. 8, pp. 36-39: integrations described; CLI/cloud integration not configured.	**Not evidenced** External pipeline governance is not an HCP integration walkthrough.

Logging clarification

For 7c, use TF_LOG for verbosity and TF_LOG_PATH to persist diagnostic logs. QATIP also shows terraform apply -debug; use the documented environment-variable approach instead. See source 4.

Additional HCP practice

The older name Terraform Cloud appears in the manuals; this matrix relates that material to HCP Terraform. Add a hosted deployment, project and workspace organisation, and a working CLI and cloud integration. General Jenkins, GitHub or Azure DevOps pipelines do not establish HCP-specific competence.

Official references

1 HashiCorp Terraform Associate 004 objectives and exam version

2 HashiCorp Associate 004 exam content list and study resources

3 HashiCorp sensitive data handling for Terraform 1.12

4 HashiCorp diagnostic logging for Terraform 1.12